

UNIVERSITY OF TECHNOLOGY, JAMAICA
SCHOOL OF COMPUTING & INFORMATION TECHNOLOGY
Programming 2: Tutorial/Practical Worksheet #4

Random Numbers in C

The rand() function generates pseudo-random numbers. Pseudorandom numbers are generated by computers. They are not truly random, because when a computer is functioning correctly, nothing it does is random. rand() will generate a value from 0 to RAND_MAX (32767).

Structure:

Shifting value + rand() % scaling value

- Shifting value: starting value of the range
- Scaling value: numbers required in the range (width of the range)
- Scaling value = last value of the range – shifting value + 1

Syntax

```
//Generate numbers between 2 and 12 inclusive  
printf("%d", 2 + rand()% + 11);
```

To generate a truly random number, the function srand(seed) from the stdlib.h is required. A seed is the starting point for a sequence of pseudo-random. If srand() is omitted, rand() seed is set by default to srand(1). Any other value for seed sets the generator to a different starting point. A preferred method for seeding is to use the system clock. The example would then become:

- `srand(time(0));`
- `srand(time(NULL));`

Require libraries

- `#include <stdlib.h>` // provides data type conversions, random numbers, memory allocation and utility functions
- `#include <time.h>` // definitions of functions to get and manipulate date and time information

Random Numbers in Python

In Python, random numbers are generated using the built-in random module. This module does not produce truly random values; instead, it generates pseudo-random numbers, which are sufficient for simulations, games, and most educational purposes.

- Before generating random numbers, you must import the module:
import random
- To generate a random integer within a specific range (inclusive):
random.randint(1, 10) # Returns a random integer between 1 and 10, including both.
- To generate a random decimal number between 0.0 and 1.0:
random.random()
- To generate a float within a specific range:

random.uniform(5.0, 10.0) # Returns a float between 5.0 and 10.0

Tutorial and Laboratory Questions

1. Give the function declaration/prototype for each of the following functions:
 - a. Function **StartProgram** that takes no arguments and returns nothing.
 - b. Function **InputValue** that takes no arguments and returns a floating-point value.
 - c. Function **OutputValue** that takes one argument, an integer value, and returns nothing.
 - d. Function **CalculateTotal** that takes three arguments, all of integer data type, and returns a floating-point value.
 - e. Function **DetermineStatus** that takes one argument, a floating-point value, and returns a character value.
2. Given the following function prototypes in **question (1)** above, write the function call for each.
3. Create the function definitions for all the prototypes found in **question (1)** above
4. Using functions, create the following solutions using C and Python source code.
 - a. Create a function that returns a random number between 1 and 6. Call the function three times.
 - b. Generate a random number between 1 and 100. If the number is even, display "Even number". If the number is odd, display "Odd number"
 - c. Generate a random number between 10 and 20. Ask the user to guess the number, then compare their guess to the random number. Display 'Correct' if they match, or 'Wrong' if they don't. If the guess is wrong, reveal the correct number.
 - d. Generate 10 random numbers between 1 and 50. Count how many numbers are: Greater than 25 and less than or equal to 25. Display both counts
 - e. Generate 3 random numbers between 1 and 9. Ask the user to enter one number. Count how many times the user's number appears. Display the following:
 - a. "No match"
 - b. "One match"
 - c. "Two matches"
 - d. "Jackpot!" (3 matches)

- f. The computer randomly chooses:
 - a. 1 = Rock
 - b. 2 = Paper
 - c. 3 = Scissors
 - d. Tasks:
 - e. Ask the user to choose a number
 - f. Display the computer's choice
 - g. Determine and display the winner
- g. Generate random scores for 10 students (0–100). Calculate and display:
 - a. Highest score
 - b. Lowest score
 - c. Average score
 - d. Count how many students passed (≥ 50)
- h. Generate a random number between 1 and 20. Allow the user 5 attempts to guess. After each guess, tell the user: "Too high" or "Too low". End the game early if the guess is correct. Display the number of attempts used

Challenge

Part 1

Computers are playing an increasing role in education. Using function, write a program that will help an elementary school student learn multiplication. Use rand to produce two positive one-digit integers. It should then type a question such as: "How much is 6 times 9?" (*Where 6 and 9 are random numbers*). The student then types the answer. Your program checks the student's answer. If it is correct, print "Very good!" and then ask another multiplication question. If the answer is wrong, print "No. Please try again." and then let the student try the same question again repeatedly until the student finally gets it right.

Part 2

The use of computers in education is referred to as computer-assisted instructions (CAI). One problem that develops in CAI environments is student fatigue. This can be eliminated by varying

the computer's dialogue to hold the student's attention. Modify the program above so the various comments are printed for each correct answer and each incorrect answer as follows:

Response to a correct answer

1. Very good!
2. Excellent!
3. Nice work!
4. Keep up the good work!

Response to an incorrect answer

1. No. Please try again.
2. Wrong. Try once more.
3. Don't give up!
4. No. Keep trying.

Use the random number generator to choose a number from 1 to 4 to select an appropriate response to each answer. Use switch statements with `printf()` statements to issue the responses.

Part 3

More sophisticated computer-aided instructions systems monitor the student's performance over a period of time. The decision to begin a new topic is often based on the student's success with previous topics. Modify the program above in the form of a quiz to count the number of correct and incorrect responses typed by the student. After the student types 10 answers, your program should calculate and display the percent.